

Dataset

We will use two different datasets, the *churn* dataset <https://archive.ics.uci.edu/dataset/563/iranian+churn+dataset> for a binary classification and the *winequality-red* dataset <https://archive.ics.uci.edu/dataset/186/wine+quality> for a categorical classification. Both datasets are available on the UCI Machine Learning repository.

The binary classification dataset contains features extracted from customers of a telecommunication company, which are classified as either churn or not. The categorical classification dataset contains chemical measurements for seven distinct qualities of a Portuguese white wine. In the former dataset, the class is indicated in the final column named "Churn", whereas for the latter, target information is provided in the last column named "quality."

Please run the code block below to download the data files.

```
import os
import zipfile
import urllib.request

# Download the two dataset files
dataset_files = {
    "churn_data.zip": "https://archive.ics.uci.edu/static/public/563/iranian+churn+dataset.zip",
    "winequality-red.csv": "https://archive.ics.uci.edu/ml/machine-learning-databases/wine-quality/winequality-red.csv"
}

for name, url in dataset_files.items():
    base, extension = os.path.splitext(name) # Get file name without extension

    if os.path.exists(name):
        print(f"File '{name}' already exists. Skipping download.")
    else:
        # Download the file
        urllib.request.urlretrieve(url, name)
        print(f"Downloaded {name} successfully.")

    # Check if it's a zip file and if it has been extracted
    if extension == ".zip":
        extracted_file = os.path.join(base + ".csv") # Expected extracted file name

        if not os.path.exists(extracted_file):
            with zipfile.ZipFile(name, 'r') as zip_ref:
                zip_ref.extractall()
            print(f"Extracted {name} successfully.")

            # Rename the extracted file
            os.rename("Customer Churn.csv", extracted_file)
        else:
            print(f"File '{extracted_file}' already extracted. Skipping extraction.")
```

```
Downloaded churn_data.zip successfully.
Extracted churn_data.zip successfully.
Downloaded winequality-red.csv successfully.
```

Task 2.1: Dataset Loading

The first task deals with the loading of the datasets. When training networks in `NumPy`, all data needs to be stored as NumPy arrays. The data should be split between input matrix $\mathbf{X} \in \mathbb{R}^{D \times N}$ and target matrix $\mathbf{T} \in \mathbb{R}^{O \times N}$. We need to **add a bias neuron to the input** so that we have $\mathbf{X} \in \mathbb{R}^{(D+1) \times N}$

Implement a function that returns both the input and the target data for a given dataset.

Note:

1. You can use `csv.reader()` to read the dataset, or rely on other methods such as `pandas`
2. Please note that in the wine dataset CSV file, all values are separated by `;`, whereas in the churn dataset, they are separated by `,`
3. For the wine dataset, convert 6 target values into a one-hot matrix so that the dimension is $\mathbb{R}^{6 \times N}$. **Be aware that the target values in the wine dataset do not start at index 0.**

```
import numpy as np
import csv

def dataset(dataset_file="winequality-red.csv", delimiter=";"):
    """
    Load a dataset from a CSV file and return the input and target matrices.
```

```

Args:
    dataset_file: Path to the dataset CSV file.
    delimiter: Delimiter used in the CSV file.

Returns:
    X: Input matrix (D+1, N) including bias neuron.
    T: Target matrix. One-hot encoded for wine dataset or float values otherwise.
"""
data = []
with open(dataset_file, 'r') as f:
    reader = csv.reader(f, delimiter=delimiter)

    header = next(reader)
    try:
        float(header[0])
        data.append(header)
    except:
        pass

    for row in reader:
        data.append(row)

print(f"Loaded dataset with {len(data)} samples")

data = np.array(data, dtype=float)

X = data[:, :-1]
y = data[:, -1]

X = X.T

N = X.shape[1]
bias = np.ones((1, N))
X = np.vstack((bias, X))

if dataset_file == "winequality-red.csv":
    classes = np.unique(y)
    class_to_index = {c: i for i, c in enumerate(classes)}

    O = len(classes)
    T = np.zeros((O, N))

    for i in range(N):
        T[class_to_index[y[i]], i] = 1

else:
    T = y.reshape(1, -1)

return X, T

```

✓ Test 1: Assert Valid Outputs

1. For the churn data, we assure that all dimensions are correct and that class labels are in $\{0, 1\}$.
2. For the wine dataset, we make sure that the dataset is in the correct dimensions, i.e., $\mathbf{X} \in \mathbb{R}^{(D+1) \times N}$ and $\mathbf{T} \in \mathbb{N}^{O \times N}$. We verify that each sample in $\mathbf{T}$ has exactly one active class (one-hot encoding).

```

X, T = dataset("churn_data.csv", ",")

assert X.shape[0] == 14, X.shape[0]
assert T.shape[0] == 1, T.shape[0]
assert np.all(T >= 0) and np.all(T <= 1)
assert T.dtype == np.float64

X, T = dataset("winequality-red.csv", ";")

assert X.shape[0] == 12, X.shape[0]
assert T.shape[0] == 6, T.shape[0]
assert np.all(np.sum(T, axis=0) == 1), "Error: Some samples are not correctly one-hot encoded!" # ensures that each column
assert T.dtype == np.float64

```

```

Loaded dataset with 3150 samples
Loaded dataset with 1599 samples

```

✓ Task 2.2: Split Training and Validation Data

The data should be split into 80% for training and 20% for validation. Implement a function that takes the full dataset (X, T) and returns (X_t, T_t, X_v, T_v) accordingly.

Write a function that splits off training and validation samples from a given dataset. **What do we need to assure before splitting?**

```
def split_training_data(X, T, train_percentage=0.8):
    """
    Split the dataset into training and validation sets.

    Args:
        X: Input matrix (D+1, N) including bias.
        T: Target matrix (0, N).
        train_percentage: Fraction of samples used for training.

    Returns:
        X_train: Training input matrix.
        T_train: Training target matrix.
        X_val: Validation input matrix.
        T_val: Validation target matrix.
    """
    N = X.shape[1]
    indices = np.random.permutation(N)

    X_shuffled = X[:, indices]
    T_shuffled = T[:, indices]

    split_idx = int(train_percentage * N)

    X_train = X_shuffled[:, :split_idx]
    T_train = T_shuffled[:, :split_idx]

    X_val = X_shuffled[:, split_idx:]
    T_val = T_shuffled[:, split_idx:]

    return X_train, T_train, X_val, T_val
```

Task 2.3: Input Data Normalization

Since the data is in different input regimes, we want to normalize the data.

Implement a function that normalizes all input data using the whitening method with given mean and standard deviation.

$$X_{norm} = \frac{X - \mu}{\sigma}$$

Compute the mean (μ) and the standard deviation (σ) for your dataset. Make sure that you handle the bias neuron x_0 correctly. Finally, normalize your input data using the implemented function.

Note: Use `numpy.mean()` and `numpy.std()` with the correct dimensions.

```
def normalize(X_train, X_val):
    """
    Normalize the training and validation input data using standardization.

    Args:
        X_train: Training input matrix (D+1, N_train) including bias.
        X_val: Validation input matrix (D+1, N_val) including bias.

    Returns:
        X_train_normalize: Normalized training input matrix.
        X_val_normalize: Normalized validation input matrix.
    """
    mean = np.mean(X_train, axis=1, keepdims=True)
    std = np.std(X_train, axis=1, keepdims=True)

    std[std == 0] = 1

    mean[0] = 0
    std[0] = 1

    X_train_normalize = (X_train - mean) / std
    X_val_normalize = (X_val - mean) / std

    return X_train_normalize, X_val_normalize
```

Gradient Descent

To train a two-layer multi-output regression network, we need to implement some functions. The network output is computed in three steps:

- Compute network activation for a batch of inputs $\mathbf{X}$: $\mathbf{A} = \mathbf{W}^{(1)}\mathbf{X}$

- Call the activation function element-wise: $\mathbf{H} = g(\mathbf{A})$. Here, we rely on the $\tanh$ function. Assure that the hidden neuron bias $\mathbf{H}_{0,:}$ is set appropriately.
- Compute the logits $\mathbf{Z}$ of the batch: $\mathbf{Z} = \mathbf{W}^{(2)} \mathbf{H}$.

Task 2.4: Activation of Output

In this task, we will implement the activation functions used in binary and categorical classification:

1. Logistic Function (for Binary Classification)

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

2. Stable Softmax Function (for Categorical Classification)

$$\tilde{\mathcal{S}}(z_o) = \frac{e^{z_o - z_{\max}}}{\sum_{o'} e^{z_{o'} - z_{\max}}}$$

```
def logistic_function(z):
    """
    Compute the logistic sigmoid activation function.

    Args:
        z: Input array containing the linear activations.

    Returns:
        y: Output after applying the sigmoid function.
    """
    return 1 / (1 + np.exp(-z))

def stable_softmax(z):
    """
    Compute the softmax activation function in a numerically stable way.

    Args:
        z: Input matrix containing the output logits (O, N).

    Returns:
        y: Probability matrix where each column sums to 1.
    """
    z_max = np.max(z, axis=0, keepdims=True)
    z_stable = z - z_max

    exp_z = np.exp(z_stable)
    sum_exp = np.sum(exp_z, axis=0, keepdims=True)

    return exp_z / sum_exp
```

Task 2.5: Network Implementation

Implement a multi-target network that computes the output matrix $\mathbf{Y}$ for a given input dataset $\mathbf{X}$ and given parameters ($\mathbf{W}^{(1)}$, $\mathbf{W}^{(2)}$) using (`numpy`) operations. Use $\tanh$ as activation function between two layers.

The function should return both the output $\mathbf{Y}$ and the output of the hidden units $\mathbf{H}$ since we will need these in gradient descent. When applying binary cross entropy loss, we apply the logistic function σ to the logits $\mathbf{Z}$ before returning it. When applying categorical cross entropy loss, we apply stable softmax $\tilde{\mathcal{S}}$ to the logits $\mathbf{Z}$ before returning it.

```
def Network(X, W1, W2, loss_type='bce'):
    """
    Compute the output of a two-layer neural network model.

    Args:
        X: Input matrix (D+1, N) including bias.
        W1: Weight matrix for the hidden layer (K, D+1).
        W2: Weight matrix for the output layer (O, K+1).
        loss_type: Type of loss function used ('bce' or 'cce').

    Returns:
        Y: Network output predictions.
        H: Hidden layer output including bias neuron.
    """
    A = W1 @ X

    H_no_bias = np.tanh(A)

    N = H_no_bias.shape[1]
```

```

bias = np.ones((1, N))
H = np.vstack((bias, H_no_bias))

Z = W2 @ H

if loss_type == 'bce':
    Y = 1 / (1 + np.exp(-Z))
elif loss_type == 'cce':
    z_max = np.max(Z, axis=0, keepdims=True)
    z_stable = Z - z_max
    exp_z = np.exp(z_stable)
    Y = exp_z / np.sum(exp_z, axis=0, keepdims=True)
else:
    raise ValueError("loss_type must be 'bce' or 'cce'")

return Y, H

```

Task 2.6: Loss Implementation

Implement a loss function that returns the Binary Cross-Entropy (BCE) loss for binary classification and the Categorical Cross-Entropy (CCE) loss for categorical classification. Note that for BCE, $O = 1$ so that both $\mathbf{Y} \in \mathbb{R}^{O \times N}$ and $\mathbf{T} \in \mathbb{N}^{O \times N}$ are considered as vectors.

$$\mathcal{J}^{\text{BCE}} = -\frac{1}{N} [\mathbf{T} \odot \log \mathbf{Y} + (1 - \mathbf{T}) \odot \log(1 - \mathbf{Y})] \vec{\mathbf{1}}_N$$

$$\mathcal{J}^{\text{CCE}} = -\frac{1}{N} \vec{\mathbf{1}}_O^T [\mathbf{T} \odot \log \mathbf{Y}] \vec{\mathbf{1}}_N$$

where $\odot$ represents the element-wise multiplication, and $\vec{\mathbf{1}}_N$ a vector containing N ones. For both losses, the mathematical expressions above can be implemented much easier, by using basic `numpy` or `scipy` functionality.

```

def loss(Y, T, loss_type):
    """
    Compute the loss between predictions and targets.

    Args:
        Y: Predicted output probabilities from the network (O, N).
        T: Target matrix (same shape as Y).
        loss_type: Type of loss function ('bce' or 'cce').

    Returns:
        L: Computed scalar loss value.
    """

    eps = 1e-12
    Y = np.clip(Y, eps, 1 - eps)

    N = Y.shape[1]

    if loss_type == 'bce':
        L = -np.sum(T * np.log(Y) + (1 - T) * np.log(1 - Y)) / N
    elif loss_type == 'cce':
        L = -np.sum(T * np.log(Y)) / N
    else:
        raise ValueError("loss_type must be 'bce' or 'cce'")

    return L

```

Task 2.7: Gradient Implementation

Implement a function that computes and returns the gradient for a given batch $(\mathbf{X}, \mathbf{T})$, the given network outputs $\mathbf{Y}$ and $\mathbf{H}$ as well as current parameters $(\mathbf{W}^{(1)}, \mathbf{W}^{(2)})$. Make sure to compute the gradient with respect to both weight matrices. Remember that we have used `tanh` as the activation function between two layers. Implement the function using the fast version provided in the lecture and make use of `numpy` operations.

```

def gradient(X, T, Y, H, W1, W2):
    """
    Compute the gradients of the loss with respect to the network weights.

    Args:
        X: Input matrix (D+1, N) including bias.
        T: Target matrix (O, N).
        Y: Predicted output from the network (O, N).
        H: Hidden layer output including bias neuron (K+1, N).

```

```

W1: Weight matrix for the hidden layer (K, D+1).
W2: Weight matrix for the output layer (O, K+1).

Returns:
    g1: Gradient with respect to W1.
    g2: Gradient with respect to W2.
"""

N = X.shape[1]

dZ2 = Y - T
g2 = (dZ2 @ H.T) / N

dH = W2.T @ dZ2

dH_no_bias = dH[1:, :]

dA1 = dH_no_bias * (1 - H[1:, :]**2)

g1 = (dA1 @ X.T) / N

return g1, g2

```

Task 2.8: Accuracy Computation

Implement a function that computes the accuracy of the provided network output and the given target values. Make sure that the implementation supports both binary as well as categorical targets.

```

def accuracy(Y, T):
    """
    Compute the classification accuracy.

    Args:
        Y: Predicted output probabilities from the network (O, N).
        T: Target matrix (same shape as Y).

    Returns:
        acc: Classification accuracy.
    """

    if Y.shape[0] == 1:
        Y_pred = (Y >= 0.5).astype(int)
        T_true = T.astype(int)

        acc = np.mean(Y_pred == T_true)

    else:
        Y_pred = np.argmax(Y, axis=0)
        T_true = np.argmax(T, axis=0)

        acc = np.mean(Y_pred == T_true)

    return acc

```

Test 2: Test Accuracy Function

Design test data and according logit values with which you can test the correctness of your accuracy function. Make sure that the accuracy will compute the correct values. Test both binary and categorical accuracy.

```

YY = np.ones((1,20)) * -5.
YY[0][15:20] = 5
assert(abs(accuracy(YY,np.zeros((1,20)))) - 0.75) < 1e-8
assert(abs(accuracy(YY,np.ones((1,20)))) - 0.25) < 1e-8

YY = np.ones((4, 20)) * -5
YY[0, 0:1] = 5
YY[1, 1:4] = 5
YY[2, 4:10] = 5
YY[3, 10:20] = 5

T0 = np.eye(4)[np.zeros(20, dtype=int)].T
T1 = np.eye(4)[np.ones(20, dtype=int)].T
T2 = np.eye(4)[np.ones(20, dtype=int) * 2].T
T3 = np.eye(4)[np.ones(20, dtype=int) * 3].T
T4 = np.eye(4)[np.array([0, 1, 1, 1, 2, 2, 2, 2, 2, 2, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3])].T

assert abs(accuracy(YY, T0) - 0.05) < 1e-8
assert abs(accuracy(YY, T1) - 0.15) < 1e-8

```

```

assert abs(accuracy(YY, T2) - 0.3) < 1e-8
assert abs(accuracy(YY, T3) - 0.5) < 1e-8
assert abs(accuracy(YY, T4) - 1.0) < 1e-8

```

Task 2.9: Training Loop

Implement gradient descent for a given number of 10'000 epochs using given input data, initial parameters ($\mathbf{W}^{(1)}$, $\mathbf{W}^{(2)}$), loss_type, number of epochs, as well as a learning rate of $\eta = 0.1$.

Make use of the normalized dataset from Task 2.3, the network from Task 2.5, the loss from Task 2.6, the gradient from Task 2.7, and the accuracy function from Task 2.8.

Make sure that you train on the training data only, and not on the validation data. In each loop, compute and store the training loss, training accuracy, validation loss and validation accuracy. At the end, return the lists of these values.

```

def train(X_train, T_train, X_val, T_val,
          W1, W2,
          epochs=100,
          lr=0.01,
          loss_type='bce'):
    """
    Train a two-layer neural network using gradient descent.

    Args:
        X_train, T_train: Training data
        X_val, T_val: Validation data
        W1, W2: Initial weights
        epochs: Number of training epochs
        lr: Learning rate
        loss_type: 'bce' or 'cce'

    Returns:
        train_loss, train_acc, val_loss, val_acc
    """
    train_loss, train_acc, val_loss, val_acc = [], [], [], []

    for epoch in range(epochs):
        Y_train, H_train = Network(X_train, W1, W2, loss_type)

        L_train = loss(Y_train, T_train, loss_type)

        g1, g2 = gradient(X_train, T_train, Y_train, H_train, W1, W2)

        W1 -= lr * g1
        W2 -= lr * g2

        train_loss.append(L_train)
        train_acc.append(accuracy(Y_train, T_train))

        Y_val, _ = Network(X_val, W1, W2, loss_type)

        L_val = loss(Y_val, T_val, loss_type)

        val_loss.append(L_val)
        val_acc.append(accuracy(Y_val, T_val))

    return train_loss, train_acc, val_loss, val_acc

```

Task 2.10: Plotting Function

Implement a function that takes four lists containing the training loss, the training accuracy, the validation loss and the validation accuracy. Plot the two losses into one plot, and the two accuracies into another plot.

```

from matplotlib import pyplot
def plot(train_loss, train_acc, val_loss, val_acc):
    """
    Plot training and validation loss and accuracy over epochs.

    Args:
        train_loss: List of training loss values per epoch.
        train_acc: List of training accuracy values per epoch.
        val_loss: List of validation loss values per epoch.
        val_acc: List of validation accuracy values per epoch.
    """
    pyplot.figure(figsize=(10,3))
    ax = pyplot.subplot(121)
    ax.plot(train_loss, "g-", label="Training set loss")

```

```

ax.plot(val_loss, "b-", label="Validation set loss")
ax.set_xlabel("Epoch")
ax.legend()

ax = pyplot.subplot(122)
ax.plot(train_acc, "g-", label="Training set accuracy")
ax.plot(val_acc, "b-", label="Validation set accuracy")
ax.set_xlabel("Epoch")
ax.legend()
pyplot.show()

```

Task 2.11: Binary Classification

1. Load the data for binary classification, using the "churn_data.csv" file.
2. Split the data into training and validation sets.
3. Normalize both training and validation input data using the function from Task 2.3.
4. Instantiate the weight matrices ($\mathbf{W}^{(1)}$, $\mathbf{W}^{(2)}$) using the Xavier method as introduced in the lecture
5. Train the network on the churn data with the learning rate of $\eta = 0.1$ for 10'000 epochs and plot the training and validation accuracies and losses.

```

X, T = dataset("churn_data.csv", delimiter=",")

X_train, T_train, X_val, T_val = split_training_data(X, T, train_percentage=0.8)

X_train, X_val = normalize(X_train, X_val)
# print the
D = X_train.shape[0] - 1
K = 10
O = T_train.shape[0]

limit_W1 = np.sqrt(6 / (D + K))
W1 = np.random.uniform(-limit_W1, limit_W1, (K, D + 1))

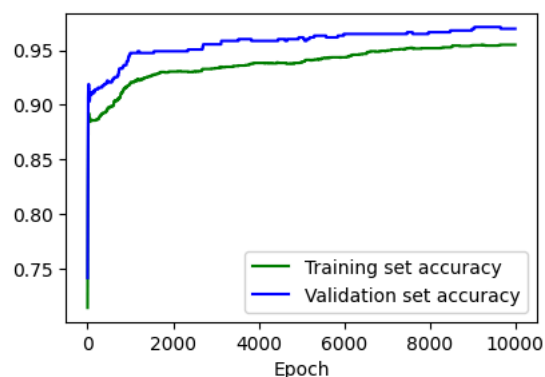
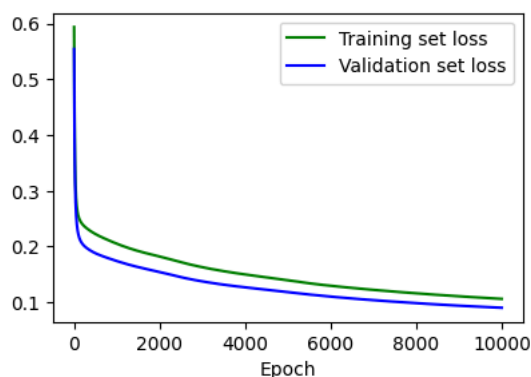
limit_W2 = np.sqrt(6 / (K + O))
W2 = np.random.uniform(-limit_W2, limit_W2, (O, K + 1))

train_loss, train_acc, val_loss, val_acc = train(
    X_train, T_train,
    X_val, T_val,
    W1, W2,
    epochs=10000,
    lr=0.1,
    loss_type='bce'
)

# Call plot
plot(train_loss, train_acc, val_loss, val_acc)

```

Loaded dataset with 3150 samples



Task 2.12: Categorical Classification

1. Load the data for categorical classification, using the "winequality-red.csv" file.
2. Split the data into training and validation sets.
3. Normalize both training and validation input data using the function from Task 2.3.
4. **How many input and output neurons do we need?** Change the number of input, hidden, and output neurons accordingly. Select an appropriate number of hidden neurons K .
5. Instantiate the weight matrices ($\mathbf{W}^{(1)}$, $\mathbf{W}^{(2)}$) using the Xavier method as introduced in the lecture

6. Train the network on the winequality-red dataset with the learning rate of $\eta = 0.1$ for 10'000 epochs and plot the training and validation accuracies and losses.

```
# load dataset
X, T = dataset("winequality-red.csv", delimiter=";")
# split dataset
X_train, T_train, X_val, T_val = split_training_data(X, T, train_percentage=0.8)
# normalize input data
X_train, X_val = normalize(X_train, X_val)

# Determine number of input, hidden, and output neurons
D = X_train.shape[0] - 1 # Input features (excluding bias)
K = 20 # Number of hidden neurons, chosen as 20 for this example
O = T_train.shape[0] # Number of output classes

# Instantiate the weight matrices (W1, W2) using Xavier method
limit_W1 = np.sqrt(6 / (D + K))
W1 = np.random.uniform(-limit_W1, limit_W1, (K, D + 1))

limit_W2 = np.sqrt(6 / (K + O))
W2 = np.random.uniform(-limit_W2, limit_W2, (O, K + 1))

# train network on our data
train_loss, train_acc, val_loss, val_acc = train(
    X_train, T_train,
    X_val, T_val,
    W1, W2,
    epochs=10000,
    lr=0.1,
    loss_type='cce'
)

# plot the results
plot(train_loss, train_acc, val_loss, val_acc)
```

Loaded dataset with 1599 samples

